

Small 2NFAs With Few Reversals Are Strictly More Powerful Than Small Sweeping 2NFAs

Jana Hamdy

March 2, 2026

Abstract

A two-way nondeterministic finite automaton (2NFA) is sweeping if its input head can change directions only on end-markers, and a 2NFA is few-reversals if its input head makes a sub-linear number of reversals in terms of input size. For every n , we exhibit a language that can be recognized by n -state few-reversal 2NFA, but requires $2^{\Omega(n)}$ states on every sweeping 2NFA (SNFA) that recognizes the language.

1 Introduction

The problem of whether P equals NP is arguably the most important problem in theoretical computer science. At its core, it is a question about the power of nondeterminism. While the P vs NP problem deals with the capabilities of Turing machines, understanding the nuances of nondeterminism at the weaker level of finite automata can provide insights into the bigger question.

In the context of two-way finite automata, the P vs NP problem has an analogy in the 2D vs 2N problem. It was first formulated by Sakoda and Sipser [SS78], and they conjecture an exponential gap between the state complexities of two-way deterministic finite automata (2DFA) and two-way nondeterministic finite automata (2NFA); that is $2D \neq 2N$.

One way to prove the conjecture is to prove that small 2NFA's are not closed under complement¹. Kapoutsis [Kap06] proved a restriction of the problem; that *sweeping* 2NFA's, where the input head can change directions only at the end-markers of the input tape, are not closed under complement. Another restriction to a 2NFA is a few-reversal 2NFA, where the input head makes a sub-linear number of direction reversals relative to the input size. A natural question is whether few-reversal 2NFA's are an *interesting* restriction; that is, does [Kap06] also imply that few-reversal 2NFA's are not closed under complement. This paper proves that's not the case; specifically, we exhibit a language that can be recognized by an n -state few-reversal 2NFA but requires $2^{\Omega(n)}$ states on any sweeping 2NFA that recognizes the same language.

The family of languages witnessing this theorem is segmented one way liveness, which is a restricted form of segmented one way liveness used in [SS78], [Kap06], and [Kap13]. The restricted form was introduced in [Kap13].

2 Preliminaries

We write $[n]$ for the set $\{1, 2, \dots, n\}$. If Σ is an alphabet, Σ^* is the set of all finite strings over Σ .

2.1 Functions and Relations

Let $V \subseteq \mathcal{P}(U)$ be a set of points in the lattice of subsets of U .

Subsets in a Lattice: For $u \in V$, the part of V below u is denoted by V_u :

$$V_u := \{v \in V \mid v \subseteq u\}.$$

Height of a Subset: The height $h_V(u)$ of u in V is defined as the length of the longest chain $\emptyset \neq u_1 \subsetneq u_2 \subsetneq \dots \subsetneq u_k$:

$$h_V(u) = k \quad \text{where} \quad \{u_1, u_2, \dots, u_k\} \subseteq V_u.$$

¹Since we know that small 2DFA's are closed under complement, proving that small 2NFAs are not closed complement implies there is a language L recognized by a small 2NFA whose complement $\bar{L}$ is not. This implies that a small 2DFA cannot recognize L ; if it did then a small 2DFA can also recognize $\bar{L}$ which contradicts the fact that 2NFAs are at least as powerful as 2DFAs.

Monotone Functions: A function $f : V \rightarrow V$ is called monotone if it respects the inclusion:

$$u \subseteq v \implies f(u) \subseteq f(v).$$

Automorphisms: A function $f : V \rightarrow V$ is an automorphism if, for all $u \in V$, its restriction to V_u is a bijection from V_u to $V_{f(u)}$.

We present some useful lemmas:

Lemma 1 *If f is an automorphism, f is also monotone*

Proof: Suppose $u \subseteq v$, then we know that $V_u \subseteq V_v$; for any $w \in V_u$, $w \subseteq u \implies w \subseteq v \implies w \in V_v$.

Note that $V_{f(u)} \subseteq V_{f(v)}$; if $y \in V_{f(u)}$, then there's $x \in V_u$ such that $f(x) = y$, since $f|_{V_u}$ is a bijection from V_u to $V_{f(u)}$. Then, $x \in V_u \subseteq V_v \implies x \in V_v \implies f(x) = y \in V_{f(v)}$ since $f|_{V_v}$ is also a bijection from V_v to $V_{f(v)}$.

Since $V_{f(u)} \subseteq V_{f(v)}$ and $f(u) \in V_{f(u)}$ by definition, $f(u) \in V_{f(v)} \implies f(u) \subseteq f(v)$ again by definition.

Lemma 2 *Every automorphism f respects heights;*

$$h_V(u) = h_V(f(u)) \quad \text{for all } u \in V.$$

Proof Consider $u \in V$. Let $h_V(u) = k$, meaning $\{\emptyset \neq u_1 \subsetneq u_2 \subsetneq \dots \subsetneq u_k\}$ is that longest chain of subsets in V_u .

Since f is a monotone (Lemma 1), $f(u_1) \subseteq f(u_2) \subseteq \dots \subseteq f(u_k)$ forms a chain in $V_{f(u)}$. Moreover, since f is a bijection, we know $f(u_1) \subsetneq f(u_2) \subsetneq \dots \subsetneq f(u_k)$, otherwise f would map k elements to less than k elements. Hence, $h_V(f(u)) \geq k$.

It remains to show that $h_V(f(u)) \leq k$. Towards a contradiction, suppose that $h_V(f(u)) > k$. Let $\emptyset \neq v_1 \subsetneq v_2 \subsetneq \dots \subsetneq v_{k+1}$ in $V_{f(u)}$ be the witnessing chain. Let $u_1, u_2, \dots, u_{k+1}$ in V_u be the pre-images under f .

We know that $u_1 \subseteq u_2 \subseteq \dots \subseteq u_{k+1}$, since f is monotone (Lemma 1); Suppose not, then there's $u_i \not\subseteq u_{i+1}$ for some i . If $x \in V_{u_{i+1}} \implies x \subseteq u_{i+1} \implies f(x) \subseteq f(u_{i+1}) \implies f(x) \in V_{f(u_{i+1})}$. Then $|V_{u_{i+1}}| \leq |V_{f(u_{i+1})}|$. However, we also know that $u_i \not\subseteq V_{u_{i+1}}$ since $u_i \not\subseteq u_{i+1}$, so Moreover, since there is a bijection from V_u to $|V_{u_{i+1}}| \leq |V_{f(u_{i+1})}|$; a contradiction to the fact that there's a bijection from $V_{u_{i+1}}$ to $V_{f(u_{i+1})}$.

We also know that $u_1 \subsetneq u_2 \subsetneq \dots \subsetneq u_{k+1}$, since $f|_{V_{u_{i+1}}}$ is a bijection. That implies there is a chain of size $k+1$ in V_u , so $h_V(u) = k+1$; contradiction.

Lemma 3 *Suppose $f : V \rightarrow V$, where $V \subseteq \mathcal{P}(U)$ is a finite set of points from the lattice of a set U . If f is injective and monotone, then it is an automorphism.*

This is presented and proved in [Kap06].

2.2 Two-way Automata

A *two-way finite automaton* (2FA) is defined as a tuple $M = (Q, \Sigma, \delta, q_0, F)$, where:

- Q is a finite set of states.
- Σ is the input alphabet.
- $\delta : Q \times (\Sigma \cup \{\#\}) \rightarrow Q \times \{L, R\}$ is the transition function, where $\#$ are end-markers.
- $q_0 \in Q$ is the initial state.
- $F \subseteq Q$ is the set of accepting states.

The head of the 2FA can move both to the left (L) and to the right (R) on the input tape, which is marked with end-markers $\#$ at both ends.

A *two-way nondeterministic finite automaton* (2NFA) is a generalization of the 2FA that can nondeterministically do multiple moves at each step, as specified by the transition function δ . In other words, a 2NFA can exhibit a set of computations (instead of one computation) for a string $z \in \Sigma^*$, and it accepts z if at least one of them falls off $\#z\#$ into q_f .

A *sweeping nondeterministic finite automaton* (SNFA) restricts the 2NFA by changing head direction only on end-markers. An input $z \in \Sigma^*$ is presented to M between two copies of the

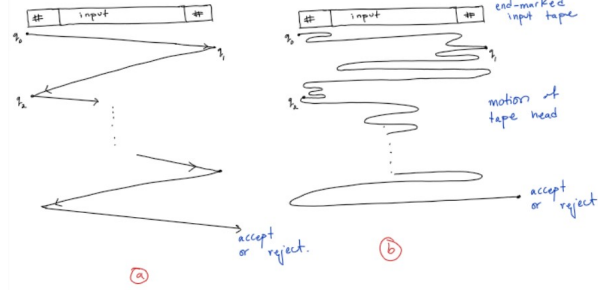


Figure 1: (a) One example computation branch of a SNFA (b) One example computation branch of a 2NFA

end-marker $\#$. The computation starts at q_s to the right of the left copy of $\#$, heading rightward. The head changes direction only when encountering an end-marker and moves to the adjacent position unless the current symbol is $\#$ and the next state is q_f .

A *few-reversal two-way nondeterministic finite automaton* (few-reversal 2NFA) is a 2NFA in which the input head makes a sub-linear number of reversals in all branches of computation. Formally, for an automaton M , if B is the set of all computation branches of M on $z \in \Sigma^*$:

$$(\exists r(n) \in o(n))(\forall z \in \Sigma^* \text{ s.t. } |z| = n)(\forall b \in B)(\text{the number of reversals in } b \leq r(n))$$

2.3 Liveness and Segmented Liveness

Given an alphabet $\Sigma_h := \mathcal{P}([h]^2)$ of all directed 2-column graphs with h nodes per column, we define a string over the alphabet to be *live* if there's a path from the leftmost column of the first symbol to the rightmost column of the last symbol of the string. For the sake of simplicity, we view any m -long string as a directed $m + 1$ -column graph (Figure 2). A string has *connectivity* $\alpha \times \beta$ if every node in α on the first symbol is connected to every node in β on the last symbol. This property of one-way-liveness (OWL_h) was first defined in [SS78].

Segmented one-way liveness (Seg-OWL) restricts this property by introducing a structure to the input strings. In Seg-OWL, the input strings are composed of segments separated by reset symbol $R := [h]^2$. Each segment consists of a unrestricted symbol between two parallel symbols Σ'_h which is of the form $[(a, a) \mid a \in \alpha]$.

We define all strings of that form:

$$\text{SEGMENTS}_h := \Sigma'_h (R \Sigma'_h \Sigma_h \Sigma'_h)^* R \Sigma'_h.$$

$$\text{Then Seg-OWL}_h := \text{SEGMENTS}_h \cap \text{OWL}_h$$

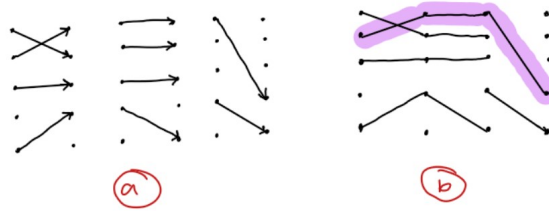


Figure 2: (a) Example symbols in Σ_5 (b) The string defined by them.

2.4 Proof Outline

First, we show that there exists a small 2NFA with few reversals that solves $\text{Seg-}\overline{\text{OWL}}_h := \{s \in \overline{\text{OWL}}_h \mid s \in \text{SEGMENTS}_h\}$; that is, all dead strings of the segmented form. Then, we show that a *sweeping* 2NFA that recognizes the language must have exponentially many states.

Fact 1 *A small 2NFA with few reversals can solve $\text{SEG-}\overline{\text{OWL}}_h$*

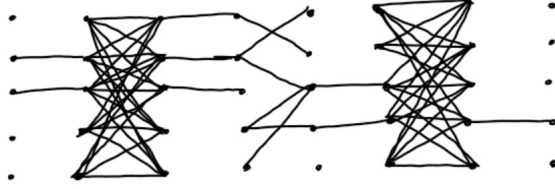


Figure 3: A string in SEGMENTS_5 with one segment

A 2NFA can solve $\text{Seg-}\overline{\text{OWL}}_h$ with $O(h^4)$ states by checking that at least one segment is dead. We can know whether a segment is dead by checking all possible paths from the first symbol in the segment to the last symbol in the segment. There are h^4 paths, and therefore $O(h^4)$ states.

The 2NFA could non-deterministically check each segment. Each branch of computation decides whether a segment is dead, and the full automaton accepts iff any of the branching computations accept. Easily, each branch makes $O(h^4)$ reversals. Therefore the 2NFA solves $\text{SEG-}\overline{\text{OWL}}_h$ with $O(1)$ reversals, in terms of n .

Theorem 1 *Every correct SNFA that recognizes $\text{SEG-}\overline{\text{OWL}}_h$ has $2^{\Omega(n)}$.*

The bulk of this paper is in proving this theorem. We do that by modifying the argument by Kapoutsis [Kap06] for SNFA's solving $\overline{\text{OWL}}_h$ to argue about SNFA's solving $\text{SEG-}\overline{\text{OWL}}_h$. This paper uses lemmas presented and proved in [Kap06].

First we present a family of hard instances of $\text{SEG-}\overline{\text{OWL}}_h$ and use the fact that any 2NFA with few reversals, M , must decide correctly on these instances to derive a combinatorial property about the number of states in M . Intuitively, these hard instances capture all the complexity of the language that make it harder for an automaton to recognize. To build these instance we need generic strings.

3 Hard Inputs and Combinatorial Property

3.1 Generic Strings

Consider any $z \in \Sigma^*$ and the set of views produced via left computations on it, starting from a set of states u :

$$\alpha_z(u) := \{q \in Q \mid (\exists r \in u)(\exists c \in \text{LCOMP}_r(z))(c \text{ hits right into } q)\}$$

We define $\beta_z : \mathcal{P}(Q) \rightarrow \mathcal{P}(Q)$ symmetrically:

$$\beta_z(u) := \{q \in Q \mid (\exists r \in u)(\exists c \in \text{RCOMP}_r(z))(c \text{ hits left into } q)\}$$

We can also define all possible sets of states reached after a left computation on z as:

$$Q_F(z) := \{\alpha_z(u) \mid u \in \mathcal{P}(Q)\}$$

Similarly,

$$Q_B(z) := \{\beta_z(u) \mid u \in \mathcal{P}(Q)\}$$

Define $a_{y,z} : Q_F(y) \rightarrow \mathcal{P}(Q)$ to be the restriction of α_z to $Q_F(y)$.

Note then that $a_{y,z}$ is a surjection from $Q_F(y)$ to $Q_F(yz)$, since $Q_F(yz)$ are exactly all sets of states that are reached via left computation on z starting from sets of states in $Q_F(y)$. This immediately implies that $|Q_F(y)| \geq |Q_F(yz)|$.

A symmetric fact also applies for right computations.

Fact 2 *For all y, z : $\alpha_{y,z}$ monotonically surjects $Q_F(y)$ to $Q_F(yz)$, so $|Q_F(y)| \geq |Q_F(yz)|$; symmetrically, $\beta_{z,y}$ monotonically surjects $Q_B(y)$ to $Q_B(zy)$, so $|Q_B(y)| \geq |Q_B(zy)|$*

Among all strings y in a language L , those with a minimum size $|Q_F(y)|$, we define as left-generic strings over L . Likewise, a string y with a minimum size $|Q_B(y)|$ among all strings in a language L are right-generic strings over L .

A string that is both left-generic and right-generic over L is called generic.

Fact 3 *If y_L is a left-generic string over language L , and y_R is a left-generic string over L , then every string $y_L x y_R \in L$ is generic.*

Intuitively, generic strings are long enough that after reading one a 2NFA has minimized the sets of states it 'pays attention to' such that any extension of the generic string that's in the language, there will be no 'merging' of sets of states after reading the extension, and the machine has effectively decided that the starting set of states Here, the merging of two sets intuitively indicates that the machine has decided that on long enough input, the starting sets do not matter

We cite one lemma from [Kap06] here that becomes useful later on.

Lemma 4 *Suppose y is generic over $L \subseteq \Sigma^*$, and $x \in \Sigma^*$. If $xy \in L$, then*

- $\alpha_{y,xy}$ is an automorphism on $Q_F(y)$, and
- $\beta_{yx,y}$ is an automorphism on $Q_B(y)$.

The proof is straightforward and proceeds by arguing that $\alpha_{y,xy}$ (and similarly $\beta_{yx,y}$) is both monotone and injective, which makes it an automorphism by Lemma 3. We provide instead some intuition into why that's the case: once the automaton has seen the generic string y , it already knows what states to "pay attention to", reading xy then doesn't decrease this number of states, and so there's a bijection between the set of states reached after reading y , and those after reading xy .

Generic strings are a useful tool for building a set of hard inputs.

3.2 Building Hard Inputs

We use the same hard input of [Kap06], [Kap13]. Here though, we select generic strings for SEG-OWL, instead of OWL.

We present them here again, along with some useful lemmas.

Fix $i = (\alpha, \beta)$ and let $L_i := L_{\alpha \times \beta}$ be the property of connecting exactly every leftmost node in α to every rightmost node in β , that is, having connectivity $\alpha x \beta$ and having the form of $\text{SEGMENTS}_h := \Sigma'_h (R \Sigma'_h \Sigma_h \Sigma'_h)^* R \Sigma'_h$.

The property L_i admits left-generic and right-generic strings by Lemma 4 since it is non-empty; for connectivity $\alpha x \beta$ we let the first and last parallel symbols be induced by α and β respectively, and have all segments be live.

For a left-generic string y_L and right-generic string y_R and the complete symbol $R = [h]^2$, then, easily, $y_L R y_R \in \text{SEGMENTS}_h$. This string is then generic over $L_{\alpha \times \beta}$ by Fact 3.

We also define $x_i := \beta x \alpha$

Lemma 5 *for all $i, j \in \mathcal{I} : j < i \implies y_i x_j y_i \in L_i$ and $j = i \implies y_i x_j y_i \in L_\emptyset$*

This is stated and proved in [Kap06].

3.3 Combinatorial Property

Consider the functions $\alpha_{y_i, x_j y_i}$ and $\beta_{y_i, x_j y_i}$ when $y_i x_j y_i$ is live; i.e when $j < i$. We know by Lemma 5 that $y_i x_j y_i \in L_i$. Hence, by Lemma 4, both $\alpha_{y_i, x_j y_i}$ and $\beta_{y_i, x_j y_i}$ are automorphisms.

On the other hand, if $j = i$ and therefore $y_i x_i y_i$ is dead, then we cannot have both functions be automorphisms. If that were true, they would be bijections, and M would not be able to distinguish between the live y_i and dead $y_i (x_i y_i)^t$ where t is the exponent that turns both bijections into identities.

However, a stronger condition is true:

Lemma 6 *if $j = i$, we can find one or two steps of M on x_i that prove one of the functions is not an automorphism.*

In other words some steps can restrict $\alpha_{y_i, x_i y_i}$ and $\beta_{y_i, x_i y_i}$ in such a way that they *cannot* be part of any steps that induces automorphisms.

This lemma is presented in and proved in [Kap06].

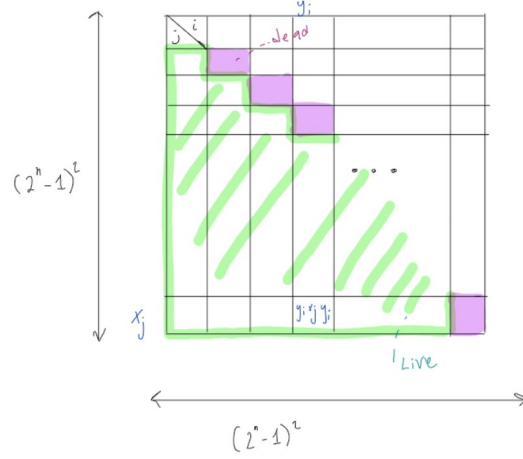


Figure 4:

4 Main Argument

Finally, using the fact that for any dead strings of the form $y_i x_j y_i$ M must exhibit one or two steps on x_i that ruin the automorphism, we know that for every $y_i x_j y_i$, there are one or two steps of M as such, illustrated in Figure 4 as a matrix of all possible i and j . We want to prove that every two cells on the matrix's diagonal have distinct elements.

Lemma 7 *No two strings $y_i x_i y_i$ and $y_j x_j y_j$ exhibit the same step or pair of steps.*

Proof: Towards a contradiction, assume $y_i x_i y_i$ and $y_j x_j y_j$ have the same witness steps $\{s_1, s_2\}$ (when $s_1 = s_2$, $\{s_1, s_2\}$ is a singleton step). Without loss of generality, assume $i < j$; otherwise, a symmetric argument applies. Consider M 's computation on $y_j x_i y_j$. After reading y_j , $\{s_1, s_2\}$ are among the steps M makes on x_i . These steps ruin the automorphism property of $\alpha_{y_j, x_i y_j}$; a contradiction to the fact that $y_i x_i y_i$ is a live string, and that $\alpha_{y_j, x_i y_j}$ must be an automorphism by Lemma 4.

Then, we know that on each of the hard instances, the diagonal cells in the matrix of Figure 1, there is a unique set of one or two steps of M that ruin the automorphism. Then the number of states of one or two steps of M must be at least as large as the number of inputs of the form $y_i x_i y_i$:

$$k^2 + \binom{k^2}{2} \geq (2n - 1)^2$$

This proves that $k = 2^{\Omega(n)}$

5 Takeaways and Conclusion

We have established an exponential separation between the state complexities of few-reversal 2NFAs and sweeping 2NFAs (SNFAs) by exhibiting a language, $\text{Seg-}\overline{\text{OWL}}_h$, that demonstrates this gap. We've shown that a few-reversal 2NFA can recognize this language. This observation implies that the challenging instances presented here cannot be used to demonstrate the non-closure under complement of few-reversal 2NFAs.

6 this is just for me

I want to summarize the last research session and what came out of it.

I also want to explore where else this could go.

Another set of hard inputs that was suggested for proving that small 2NFAs are strictly more powerful than 2NFAs with few reversals consists of a generic string d for the property of being dead that is prefixed and suffixed by a generic string g for the property of being live.

we look at blocks of the form $d s d$, where d is sipser's input. That is, it is a dead string such that if

References

- [Kap06] Christos A Kapoutsis. Small sweeping 2nfas are not closed under complement. In *International Colloquium on Automata, Languages, and Programming*, pages 144–156. Springer, 2006.
- [Kap13] Christos A Kapoutsis. Nondeterminism is essential in small two-way finite automata with few reversals. *Information and Computation*, 222:208–227, 2013.
- [SS78] William J Sakoda and Michael Sipser. Nondeterminism and the size of two way finite automata. In *Proceedings of the tenth annual ACM symposium on Theory of computing*, pages 275–286, 1978.